

UNIDAD 4

ACTIVIDAD PRÁCTICA

PROYECTO: ANÁLISIS DE SENTIMIENTO Y TENDENCIAS EN REDES SOCIALES - INTEGRACIÓN CON APIS Y ANÁLISIS AVANZADO

ÍNDICE

ORGANIZACIÓN DE LA ACTIVIDAD PRÁCTICA.....	3
DESCRIPCIÓN DE LA ACTIVIDAD PRÁCTICA	3
RECOMENDACIONES E INDICACIONES PARA LA ENTREGA.....	7
RÚBRICA DE CORRECCIÓN	7
COMO REALIZAR LA ENTREGA.....	9

ORGANIZACIÓN DE LA ACTIVIDAD PRÁCTICA

Proyecto: análisis de sentimiento y tendencias en redes sociales – Integración con APIs y análisis avanzado	
Tipo de tarea	Individual
Entregables	Repositorio de Github, Google Colab o documento comprimido

DESCRIPCIÓN DE LA ACTIVIDAD PRÁCTICA

En esta entrega, los alumnos deberán ampliar la clase DataExtractor que se desarrolló en la Unidad 2, incorporando la funcionalidad de conectarse a la API de Twitter mediante RapidAPI. El entregable deberá demostrar la correcta integración de datos en tiempo real con la aplicación de técnicas avanzadas de análisis textual, correspondientes a las Unidades 3 y 4. Se exige que:

1. Integración con la API de Twitter:
 - Implementar un nuevo método denominado load_data_api() que realice lo siguiente:
 1. Establezca la conexión a la API de Twitter utilizando RapidAPI, mediante el uso de solicitudes HTTP (por ejemplo, con la librería requests). Se puede utilizar cualquier API de Twitter (tanto oficial como con RapidAPI). Se recomienda utilizar una gratuita como por ejemplo <https://rapidapi.com/alexanderxbx/api/twitter-api45>
 2. Utilice parámetros de búsqueda (por ejemplo, palabra clave, fecha, número máximo de resultados) definidos de forma explícita en el código.
 3. Verifique el éxito de la petición y extraiga el contenido en formato JSON.
 4. Almacene el conjunto de datos extraído en el atributo self.data de la clase. Además, se debe almacenar en un archivo para utilizar posteriormente en el resto de los métodos
 5. Nota: Las claves de autenticación deben gestionarse de forma segura (por ejemplo, mediante variables de entorno) y no deben incluirse de forma explícita en el código compartido.
2. Script/Notebook de Modelado y Análisis Avanzado:
 - Implementación de modelos LDA con gensim para el descubrimiento de tópicos.
 - Código para análisis de sentimiento (TextBlob y/o spaCy).
 - Técnicas de parsing de documentos utilizando NLTK o spaCy.
 - Métodos extractivos tradicionales para resumir textos.
3. Resultados de Análisis Avanzado:

- Reporte e interpretación de los tópicos extraídos.
- Gráficos y tablas que muestren la distribución de sentimientos y ejemplos de árboles sintácticos.
- Ejemplos de resúmenes generados del corpus.

4. Documentación Detallada:

- Explicación de cada técnica aplicada, justificación metodológica y resultados obtenidos.
- Instrucciones claras para reproducir los experimentos.

Se deberá centralizar todos los pasos en la clase de Python generada en el entregable anterior. Se deberá completar con unos métodos nuevos referentes a esta unidad. Se podrá hacer uso de otras funciones, clases, métodos, etc., siempre y cuando se acabe centralizando todo en esta clase:

class DataExtractor:

```
def __init__(self, source: str = None, chunksize: int = 100000):
```

```
    """
```

Inicializa el extractor.

Parámetros:

source: Ruta al archivo local de datos (CSV o JSON). En esta entrega, tras extraer los datos vía API, se utilizará este archivo para cargar los datos.
chunksize: Tamaño de cada chunk para la lectura en caso de archivos grandes.

```
    """
```

```
self.source = source
```

```
self.data = None
```

```
self.chunksize = chunksize
```

```
def load_data_api(self, query: str, max_results: int = 100, output_file: str = "tweets_from_api.csv") -> pd.DataFrame:
```

```
    """
```

Conecta con la API de Twitter a través de RapidAPI para extraer tweets en tiempo real, y almacena los resultados en un archivo local.

Parámetros:

query: Palabra clave o hashtag a buscar.
max_results: Número máximo de tweets a extraer.
output_file: Nombre del archivo local donde se almacenarán los datos extraídos.

Proceso:

- Realiza la llamada a la API utilizando requests.
- Verifica el código de estado y extrae los datos (asumiendo que la respuesta tiene una clave 'data' con la lista de tweets).
- Adapta los nombres de columnas mínimas (por ejemplo, 'author_id' a 'user_name', 'created_at' a 'date').
- Guarda los datos en el archivo local (en formato CSV).

Devuelve:

DataFrame con los datos extraídos y almacenados.

```
    """
```

```
return self.data

def load_data(self) -> pd.DataFrame:
    """
    Carga los datos desde el archivo local especificado en self.source.

    Devuelve:
        DataFrame con los datos cargados.
    """

    return self.data

def clean_text(self, text: str) -> str:
    """
    Limpia y normaliza el texto.

    Pasos sugeridos:
    - Convertir a minúsculas.
    - Eliminar URLs.
    - Conservar hashtags y eliminar espacios redundantes.

    Devuelve:
        El texto limpio.
    """

    return text

def extract_hashtags(self, text: str) -> list:
    """
    Extrae y devuelve una lista de hashtags presentes en el texto.

    Implementación:
    - Utiliza expresiones regulares para encontrar palabras que comiencen con '#'.

    Devuelve:
        Lista de hashtags.
    """

    return hashtags

def analytics_hashtags_extended(self) -> dict:
    """
    Realiza un análisis avanzado de hashtags sobre los datos cargados en self.data.

    Pasos:
    1. Normaliza la columna 'text' y almacena el resultado en 'clean_text'.
    2. Extrae hashtags de 'clean_text' y crea la columna 'hashtags'.
    3. Convierte la columna 'date' a tipo datetime (solo fecha).
    4. Explota la columna 'hashtags' para contar cada ocurrencia.
```

5. Calcula:

- Frecuencia global de cada hashtag ('overall').
- Frecuencia de hashtags por usuario ('by_user').
- Evolución de la frecuencia por fecha ('by_date').

Devuelve:

Diccionario con DataFrames en claves: 'overall', 'by_user', 'by_date'.

"""

return {'overall': overall, 'by_user': by_user, 'by_date': by_date}

def generate_hashtag_wordcloud(self, overall_df: pd.DataFrame = None, max_words: int = 100, figsize: tuple = (10, 6)) -> None:

"""

Genera y muestra una WordCloud basada en el análisis global de hashtags.

Parámetros:

overall_df: DataFrame con columnas ['hashtags', 'frequency']. Si es None, se calcula.

max_words: Número máximo de palabras en la WordCloud.

figsize: Tamaño de la figura.

"""

def model_topics(self, num_topics: int = 5, passes: int = 10) -> list:

"""

Aplica el modelo LDA para descubrir tópicos en el corpus.

Pasos:

1. Asegurarse de que la columna 'clean_text' existe (se debe llamar previamente a clean_text).
2. Tokeniza la columna 'clean_text' (división simple en palabras).
3. Crea un diccionario y un corpus (bag-of-words) a partir de los tokens.
4. Entrena el modelo LDA con los parámetros especificados.
5. Extrae y muestra los tópicos en formato lista (cada tópico es una lista de palabras).

Devuelve:

Lista de tópicos, por ejemplo: [['word1', 'word2', ...], ['word3', 'word4', ...], ...]

"""

return topics

def analyze_sentiment(self, method: str = 'textblob') -> pd.DataFrame:

"""

Analiza el sentimiento de cada tweet utilizando el método especificado.

Parámetros:

method: 'textblob' o 'spacy'. Si se elige 'spacy', se usará spacytextblob.

Proceso:

- Para cada 'clean_text', calcula la polaridad y subjetividad.
- Almacena los resultados en las columnas 'sentiment_polarity' y 'sentiment_subjectivity'.

Devuelve:

DataFrame actualizado con las nuevas columnas de sentimiento.

"""

return self.data

def parse_and_summarize(self, summary_ratio: float = 0.3) -> str:

"""

Realiza un análisis de parsing y genera un resumen extractivo del corpus.

Pasos:

1. Concatena todos los textos limpios.
2. Divide el texto concatenado en oraciones.
3. Calcula una puntuación para cada oración basándose en la frecuencia de palabras (excluyendo stopwords).
4. Selecciona las oraciones con mayor puntuación según el ratio especificado.
5. Devuelve el resumen formado por las oraciones seleccionadas, manteniendo el orden original.

Parámetros:

summary_ratio: Proporción de oraciones a retener (ej. 0.3 para el 30%).

Devuelve:

Un string con el resumen generado.

"""

return summary.

RECOMENDACIONES E INDICACIONES PARA LA ENTREGA

Se recomienda:

- Utilizar notebooks, sea en VSCode, Jupyter o PyCharm o Google Colab
- Guardar todo en un repositorio de Github para usar como base en futuras entregas
- ¡OJO! Muchas APIs devuelven un número muy reducido de tweets, habrá que concatenar llamadas. En caso de esto suponer un problema, se puede utilizar el dataset del entregable de la unidad 2 sobre Bitcoin.
- Asegurarse de gestionar la autenticación de forma segura (por ejemplo, utilizando variables de entorno o archivos de configuración que no se suban al repositorio).
- Genera un archivo de markdown o txt para la documentación
- Recuerda, para archivos de gran tamaño es útil utilizar "chunksize" en Pandas y desactivar el modo "low_memory"

RÚBRICA DE CORRECCIÓN

Criterio	Excelente	Satisfactorio	No Satisfactorio	Insuficiente
----------	-----------	---------------	------------------	--------------

Conexión a la API y Extracción de Datos	De 1.6 a 2 puntos - Implementación robusta del método <code>load_data_api()</code> con conexión segura a la API de Twitter (RapidAPI). - Extracción completa y coherente de tweets - Manejo de errores	De 1.1 a 1.5 puntos - Conexión funcional a la API y extracción mayoritariamente correcta de tweets.	De 0.6 a 1 punto - Conexión incompleta o con errores en la extracción.	De 0 a 0.5 puntos - No se logra establecer conexión o se extraen datos irrelevantes.
Almacenamiento y Preprocesamiento	De 1.6 a 2 puntos - Implementación precisa de la transformación de los datos extraídos a un formato tabular (CSV o JSON) en memoria. - Limpieza y normalización de textos aplicadas de forma completa.	De 1.1 a 1.5 puntos - Preprocesamiento funcional, con limpieza y normalización mayoritariamente correcta. - Almacenamiento en formato estructurado sin mayores problemas.	De 0.6 a 1 punto - Preprocesamiento incompleto o con errores significativos que afectan la calidad del análisis. - Almacenamiento parcial o con problemas.	De 0 a 0.5 puntos - No se logra limpiar ni normalizar adecuadamente el texto. - Almacenamiento erróneo o inexistente del dataset extraído.
Análisis de tópicos y sentimiento	De 1.6 a 2 puntos - Implementación robusta de LDA con gensim; tópicos identificados de forma clara y precisa - Análisis de sentimiento completo y preciso (TextBlob o spaCy); resultados claros y bien interpretados; visualizaciones efectivas.	De 1.1 a 1.5 puntos - Implementación funcional de LDA; identificación mayoritariamente correcta de tópicos - Análisis funcional con resultados mayoritariamente correctos; visualizaciones adecuadas pero limitadas.	De 0.6 a 1 punto - Implementación parcial o con errores; tópicos identificados de forma poco clara - Análisis parcial o con errores; resultados difíciles de interpretar; visualizaciones poco claras.	De 0 a 0.5 puntos - No se implementa LDA correctamente; tópicos irrelevantes o inexistentes - No se logra realizar el análisis de sentimiento; resultados erróneos o inexistentes; falta de visualización.
Integración de Técnicas Avanzadas (parsing y resumen)	De 1.6 a 2 puntos - Aplicación efectiva de técnicas de parsing (NLTK y/o spaCy); estructuración precisa de textos complejos	De 1.1 a 1.5 puntos - Aplicación funcional del parsing; estructura mayoritariamente correcta	De 0.6 a 1 punto - Técnicas de parsing con errores; estructura poco clara - Métodos de resumen incompletos o con	De 0 a 0.5 puntos - No se implementan técnicas de parsing; estructura incorrecta o inexistente

	- Implementación completa de métodos extractivos de resumen; resúmenes claros, concisos y representativos	- Implementación funcional de métodos de resumen; resúmenes mayoritariamente correctos	errores; resúmenes poco claros	- No se implementan métodos de resumen; resúmenes irrelevantes o inexistentes
Presentación y Documentación	De 1.6 a 2 puntos - Presentación clara, profesional y bien estructurada del código, resultados y flujo del proyecto. - Documentación completa que facilita la reproducibilidad, con capturas y diagramas.	De 1.1 a 1.5 puntos - Presentación adecuada de los resultados y código, aunque podría ser más detallada. - Documentación suficiente, pero con ciertos aspectos mejorables.	De 0.6 a 1 punto - Presentación confusa o incompleta de resultados y código. - Documentación insuficiente o poco detallada que afecta la reproducibilidad.	De 0 a 0.5 puntos - Presentación desorganizada o inexistente; falta total de documentación que impida reproducir el proceso.

COMO REALIZAR LA ENTREGA

A modo de guión o proceso sobre cómo realizar la entrega, se recomienda seguir el siguiente proceso:

- Modificación de la Clase Base:
 - Parte del código de la clase DataExtractor de la Unidad 2.
 - Completa los nuevos métodos, por ejemplo, `load_data_api()`, que se conecte a la API de Twitter mediante RapidAPI, realice la petición con los parámetros definidos y almacene el resultado en `self.data` (y opcionalmente guarde una copia local en CSV o JSON).
- Procesamiento y Análisis Avanzado:
 - Aplica los métodos ya existentes (`clean_text`, `extract_hashtags`, `analytics_hashtags_extended`, `generate_hashtag_wordcloud`) a los datos extraídos.
 - Integra nuevas técnicas de modelado y análisis, como LDA para descubrimiento de tópicos, análisis de sentimiento (con TextBlob o spaCy), parsing (usando NLTK o spaCy) y generación de resúmenes de textos. Estos pueden implementarse como métodos adicionales o en un pipeline independiente, pero deben documentarse y vincularse al proceso global.
- Documentación:
 - Incluye un archivo README.md que contenga:
 - Descripción de la fuente de datos (la API de Twitter a través de RapidAPI) y los parámetros de búsqueda.
 - Explicación detallada de la metodología de extracción, preprocesamiento, análisis avanzado y visualización.
 - Instrucciones claras para reproducir el proceso en un entorno local.

- Diagramas y capturas de pantalla que ilustren el flujo del proyecto y los resultados obtenidos.
- Subida y Entrega:
 - Sube todo el código, documentación, y capturas a un repositorio de GitHub, en un archivo ZIP, o un enlace a Google Colab.
 - Asegúrate de que las claves de autenticación se gestionen de forma segura (por ejemplo, utilizando variables de entorno) y no se incluyan en el repositorio público.